Tecnológico Nacional de México Instituto Tecnológico de Tijuana

Materia: Gestión de proyectos de software

Grupo: SC7C

Carrera: Sistemas Computacionales

Periodo: Agosto - Diciembre 2026

Docente: Maria Guadalupe Rodriguez Lopez

Alumnos:
González Cristóbal Omar
Garcia Luna Ricardo Daniel
Cuevas Marquez Pablo Angel
Perez Flores Andres Manuel

Unidad: 3

Actividad: Evidencia del desarrollo del software

Fecha: 31/10/2026

Fecha en que se tomaron las capturas de ejecución: 30/10/2026

Índice

Resumen de la evidencia.....2
1. Código fuente desarrollado.....3
1.1 Backend (API) – Angel.....3
1.2 Frontend (página web) – Omar.....3
1.3 Base de datos – Daniel.....4
1.4 Fragmento: estado de disponibilidad.....4
2. Base de datos.....5
3. API en ejecución.....8
4. Interfaces y prototipos.....13
4.1 Wireframes de las seis pantallas.....13
4.2 Interfaz funcionando con datos reales.....20
5. Diagramas y otros artefactos técnicos.....22
6. Pruebas realizadas.....24
7. Alcance actual y pendientes.....25
7.1 Qué funciona hoy.....25
7.2 Qué sigue en la Etapa III.....25

Resumen de la evidencia

Este documento reúne la evidencia del desarrollo del software de Nexus PC, una tienda en línea ficticia de componentes de computadora. El avance verificable de esta etapa es el flujo de consulta del catálogo de extremo a extremo: la base de datos SQL Server con 30 productos, una API en Node.js y Express que los entrega y una página web que muestra el catálogo por categorías con datos reales.

La siguiente tabla indica dónde se encuentra cada evidencia que pide la lista de la Etapa II.

Evidencia solicitada	Dónde está en este documento
Código fuente desarrollado	Sección 1: tabla de módulos, rutas, responsables y fragmento de código.
Base de datos	Sección 2: diagrama entidad-relación, tablas y scripts.
Capturas de pantalla de la ejecución o avances	Sección 3 (API en ejecución) y sección 4 (interfaz con datos reales); sección 6 las enlaza con cada prueba.
Interfaces o prototipos	Sección 4: wireframes de las seis pantallas y capturas de la interfaz funcionando.
Diagramas u otros artefactos técnicos	Sección 2 (diagrama entidad-relación) y sección 5 (flujo de compra y artefactos del repositorio).

1. Código fuente desarrollado

El código está en la carpeta 01_Codigo_Fuente del repositorio (backend y frontend) y en 03_Base_de_Datos (scripts SQL). La tabla indica la ruta de cada módulo, su responsable y su función. Las líneas son las del archivo tal como está en el repositorio.

1.1 Backend (API) – Angel

Módulo (01_Codigo_Fuente/backend/)	Función	Líneas
servidor.js	Punto de entrada: crea el servidor Express, conecta las rutas, sirve el frontend y prueba la conexión.	72
db/conexion.js	Pool de conexiones a SQL Server con datos del archivo .env.	110
db/probarConexion.js	Script npm run probar-conexion: comprueba la conexión y cuenta los productos.	66
rutas/categorias.js	GET /api/categorias	42
rutas/productos.js	GET /api/productos y filtro categoria_id	62
rutas/productoPorId.js	GET /api/productos/:id con especificaciones	74
utilidades/errores.js	Respuestas de error con formato único (400, 404, 500).	50
utilidades/formato.js	Conversión a snake_case y cálculo del estado según el stock.	82
utilidades/validaciones.js	Validación de números enteros.	20

1.2 Frontend (página web) – Omar

Módulo (01_Codigo_Fuente/frontend/)	Función	Líneas
index.html	Estructura de la página: encabezado, menú de categorías, lista de productos y carrito.	50
css/estilos.css	Diseño visual: tres columnas en escritorio, dos en móvil y botones táctiles de 44 px.	113
js/api.js	Comunicación con la API (/api/categorias y /api/productos); modo de datos de ejemplo.	39
js/app.js	Lógica del catálogo: tarjetas de producto, selección de categoría, estados y panel del carrito.	284
js/carrito.js	Carrito en memoria del navegador con control de cantidades según el stock.	40
js/datos-ejemplo.json	Los 30 productos del catálogo, para trabajar sin servidor.	–

1.3 Base de datos – Daniel

Archivo (03_Base_de_Datos/)	Función	Líneas
NexusPC.sql (v0.2)	Crea la base NexusPC: 5 tablas, 7 categorías, 30 productos y sus especificaciones.	348
Crear_Usuario_API.sql	Crea el usuario nexus_app con permisos mínimos de lectura y escritura sobre NexusPC.	71

1.4 Fragmento: estado de disponibilidad

La API calcula el estado de cada producto a partir de su stock (archivo utilidades/formato.js): sin piezas es Agotado, de 1 a 3 piezas es Pocas piezas y con más de 3 es Disponible.

```
function calcularEstado(stock) {
  if (stock <= 0) return 'Agotado';
  if (stock <= 3) return 'Pocas piezas';
  return 'Disponible';
}
```

2. Base de datos

La base NexusPC está en SQL Server (versión 2022 Express) y se administra con SQL Server Management Studio. La versión vigente del script es la v0.2, resultado de las solicitudes de cambio CAM-001 (correo obligatorio en el pedido) y CAM-002 (claves de especificaciones unificadas y catálogo de 30 productos).

Tabla	Contenido
Categorias	Las 7 categorías del catálogo: Procesadores, Tarjetas madre, Memoria RAM, Almacenamiento, Tarjetas de video, Fuentes de poder y Gabinetes.
Productos	30 productos con nombre, marca, descripción, precio, stock e imagen, ligados a su categoría.
EspecificacionesProducto	Datos técnicos por producto en pares clave-valor (socket, consumo, núcleos, tipo de memoria y otros), que usarán las reglas de compatibilidad.
Pedidos	Datos del comprador (nombre, correo obligatorio y teléfono opcional), fecha y total.
DetallePedido	Productos, cantidades y precio unitario de cada pedido.

Los precios son de referencia, en pesos mexicanos y con fines académicos. La API se conecta con el usuario nexus_app, que solo tiene permisos de lectura y escritura sobre NexusPC y no puede borrar tablas.

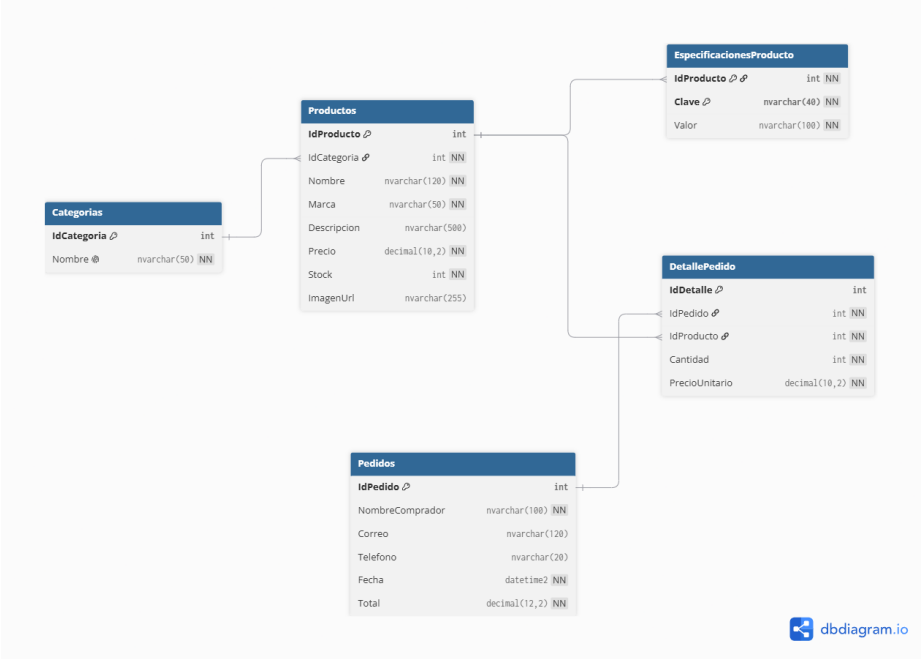


Figura 1. *Diagrama entidad-relación v0.2 de Nexus PC (elaborado en dbdiagram.io): 5 tablas y sus relaciones.*

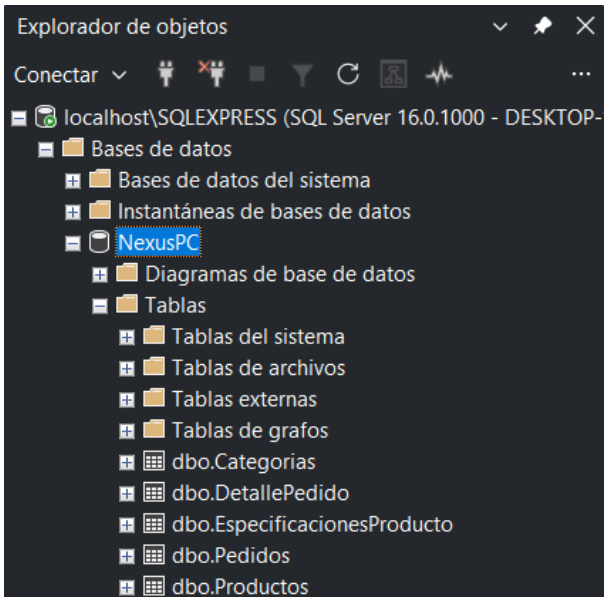


Figura 2. *SQL Server Management Studio: la base NexusPC con sus 5 tablas (Categorias, DetallePedido, EspecificacionesProducto, Pedidos y Productos).*

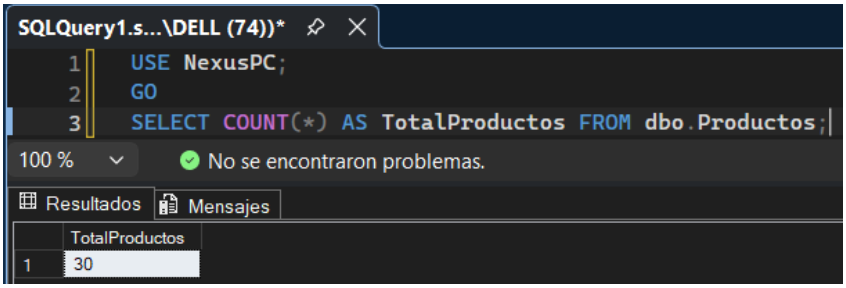


Figura 3. *Consulta SELECT COUNT(*) sobre dbo.Productos: la base contiene 30 productos.*

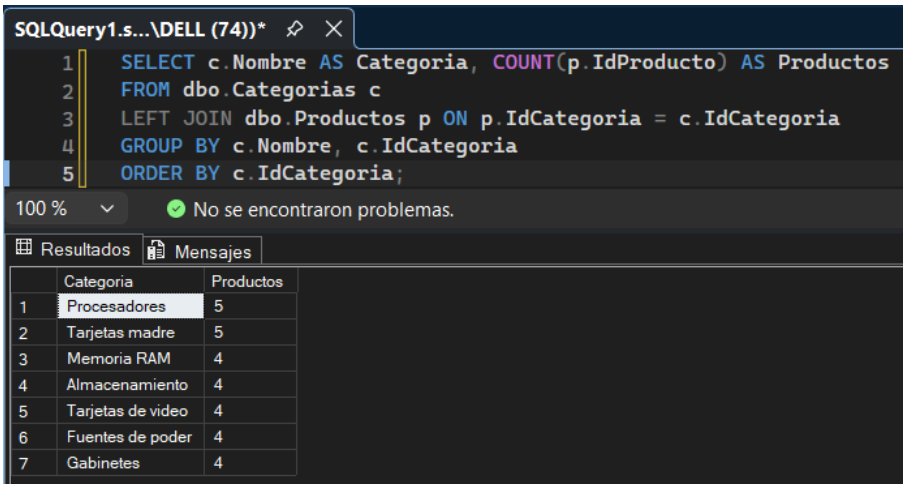


Figura 4. *Productos por categoría: 5 procesadores, 5 tarjetas madre y 4 productos en cada una de las demás categorías (30 en total).*

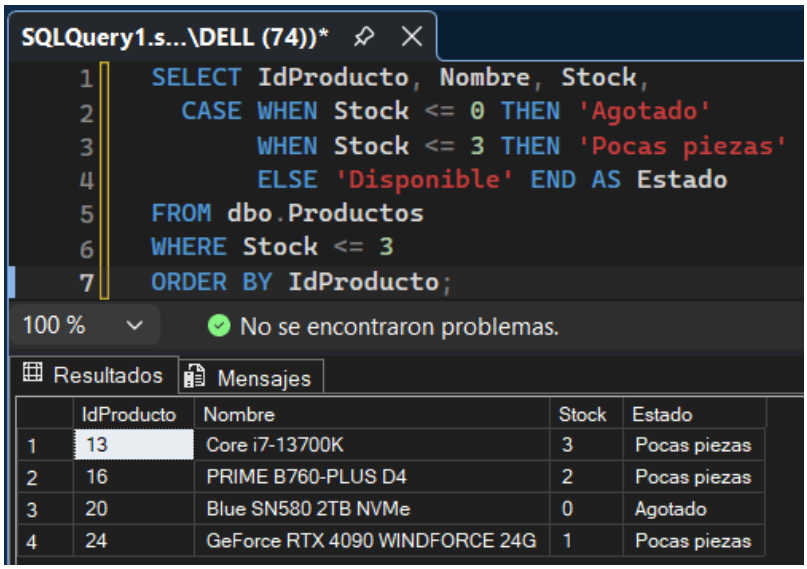


Figura 5. *Productos con stock de 3 piezas o menos: los ids 13, 16 y 24 con Pocas piezas y el id 20 Agotado, como define la regla de estado.*

3. API en ejecución

La especificación de la API (versión 0.2) define seis rutas. En esta etapa están implementadas y probadas las tres de consulta del catálogo; las demás se desarrollan en la Etapa III.

Ruta	Función	Estado
GET /api/categorias	Lista las categorías con su número de productos.	Implementada y probada
GET /api/productos	Lista los productos; admite el filtro categoria_id.	Implementada y probada
GET /api/productos/:id	Detalle de un producto con especificaciones.	Implementada y probada
POST /api/pedidos	Registra un pedido.	Especificada; Etapa III
GET /api/pedidos	Consulta de pedidos protegida con clave.	Especificada; Etapa III
POST /api/compatibilidad	Revisa la compatibilidad entre componentes.	Especificada; Etapa III

Las pruebas se hicieron el 30 de octubre de 2026 con Node.js 24, SQL Server 2022 Express y el navegador Chrome. Las capturas siguen el orden en que se ejecutaron.

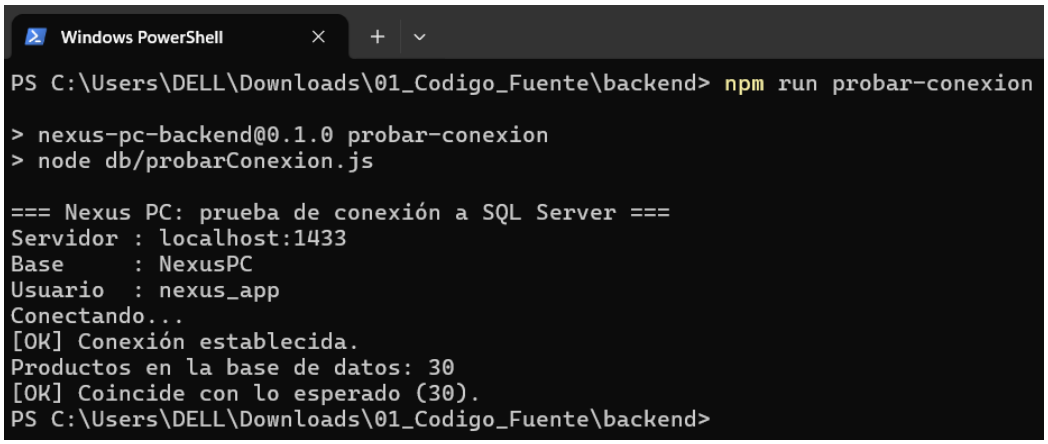


Figura 6. *Prueba de conexión (npm run probar-conexion) con el usuario nexus_app: la base NexusPC responde y contiene 30 productos, como se esperaba.*

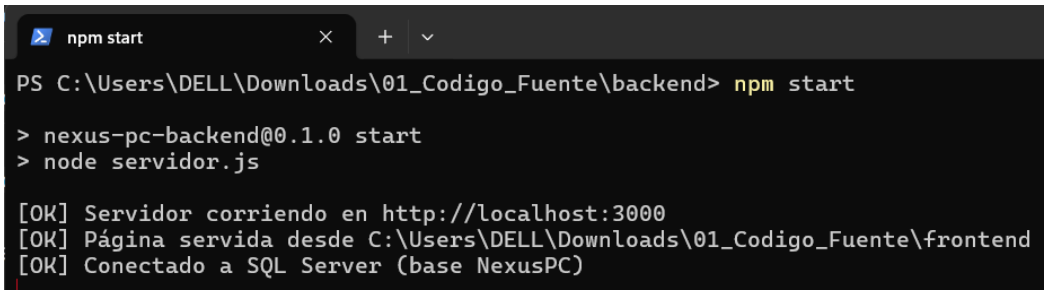


Figura 7. *Servidor iniciado con npm start en http://localhost:3000: sirve la página del frontend y se conecta a SQL Server.*

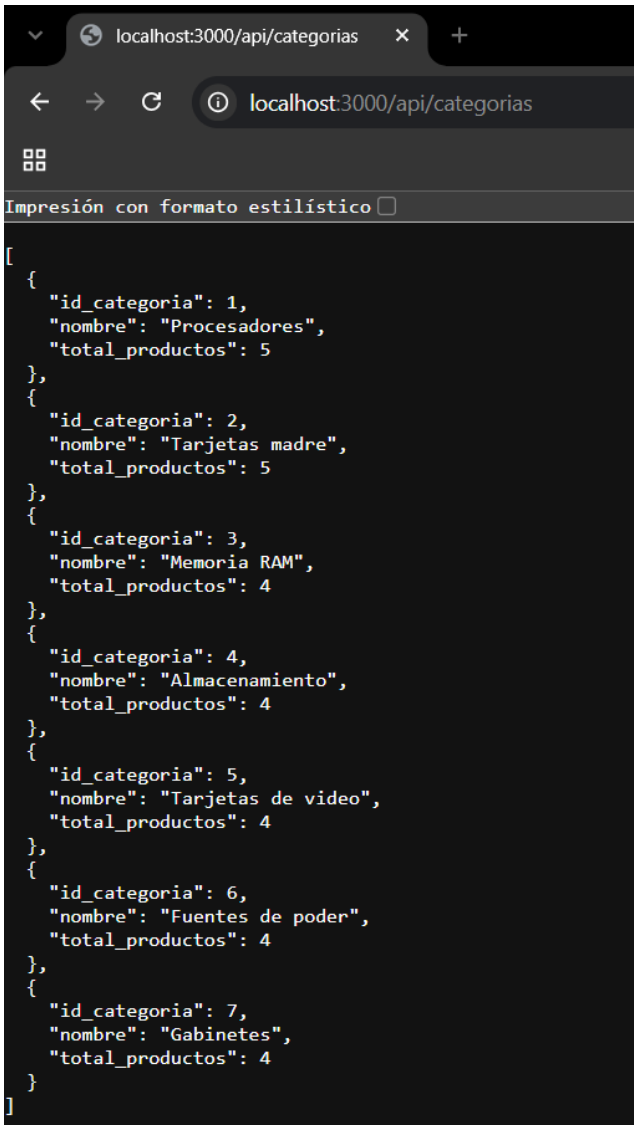


Figura 8. *GET /api/categorias: 7 categorías con el número de productos de cada una (5, 5, 4, 4, 4, 4 y 4).*

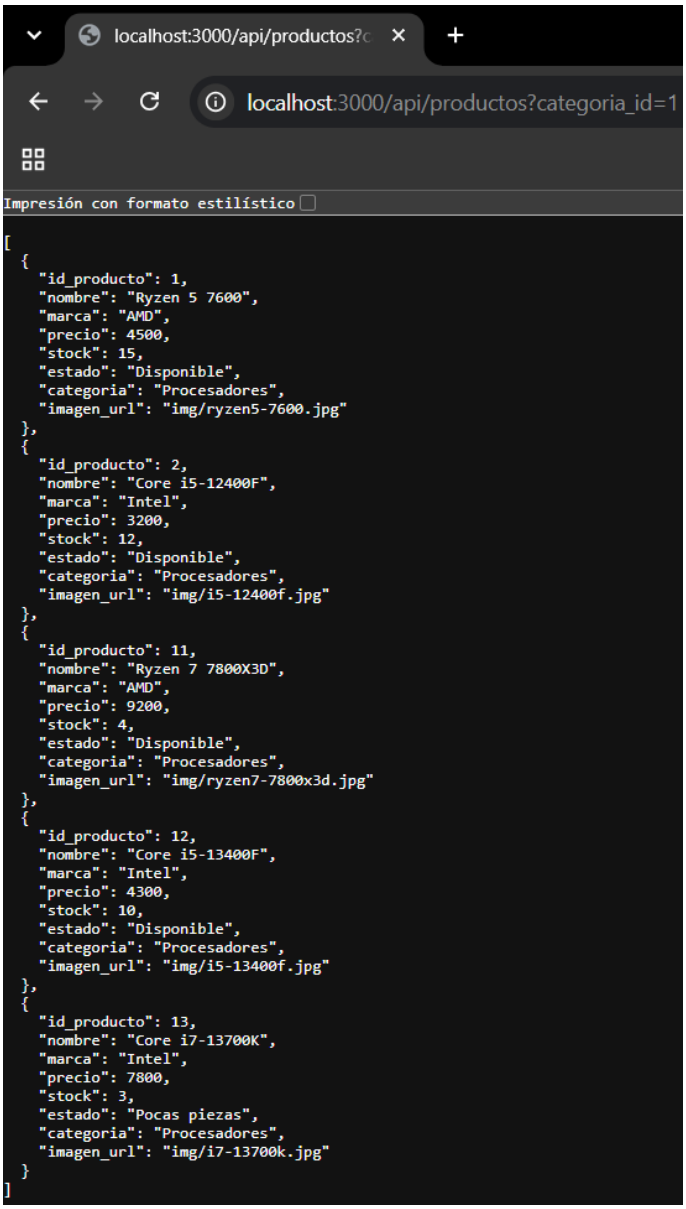


Figura 9. *GET /api/productos?categoria_id=1: los 5 procesadores. El producto 13, con stock 3, aparece con el estado Pocas piezas.*

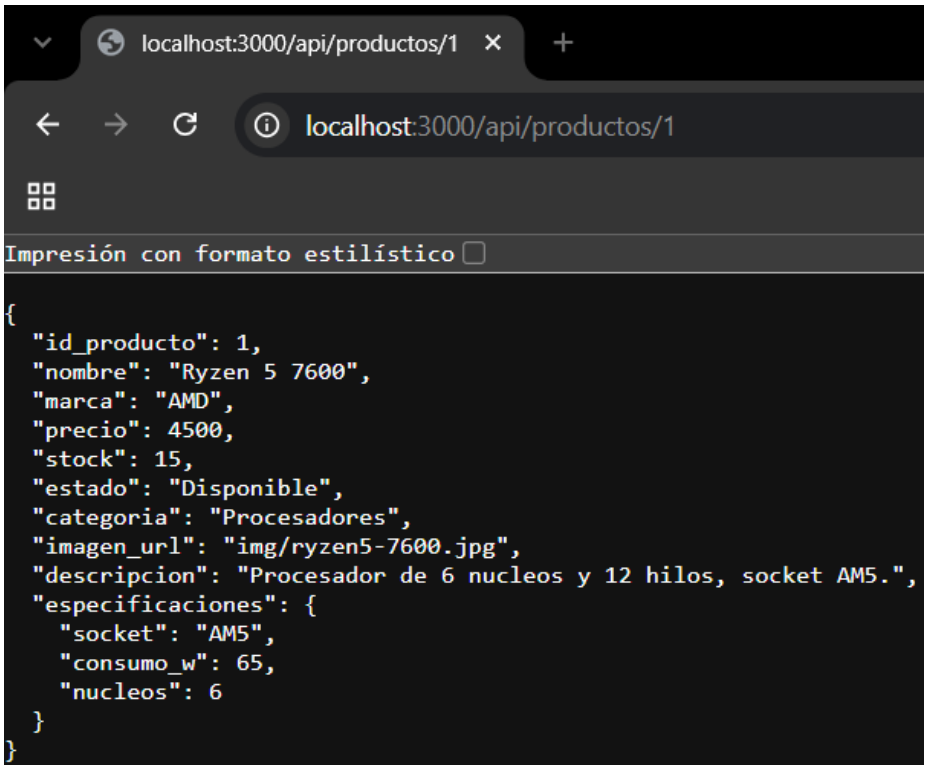


Figura 10. *GET /api/productos/1: detalle del Ryzen 5 7600 con descripción y especificaciones (socket AM5, consumo de 65 W y 6 núcleos).*

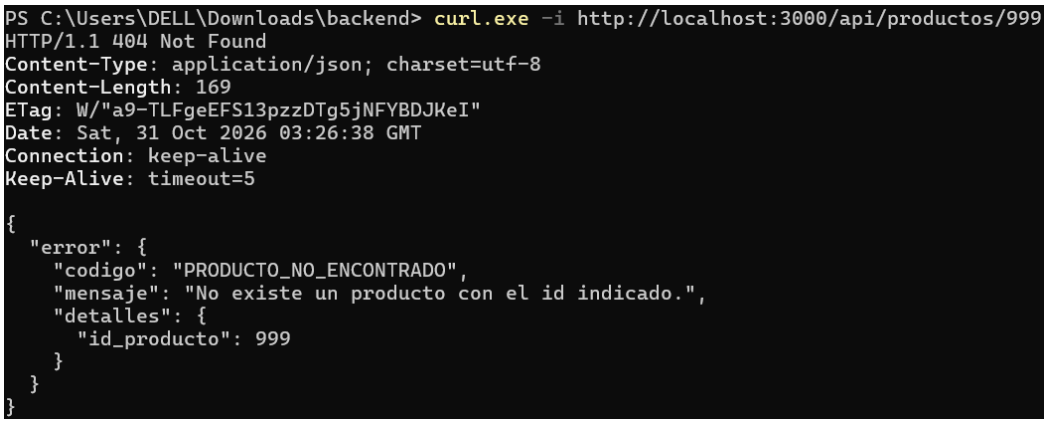


Figura 11. *GET /api/productos/999: respuesta HTTP 404 con el formato de error de la especificación (código PRODUCTO_NO_ENCONTRADO).*

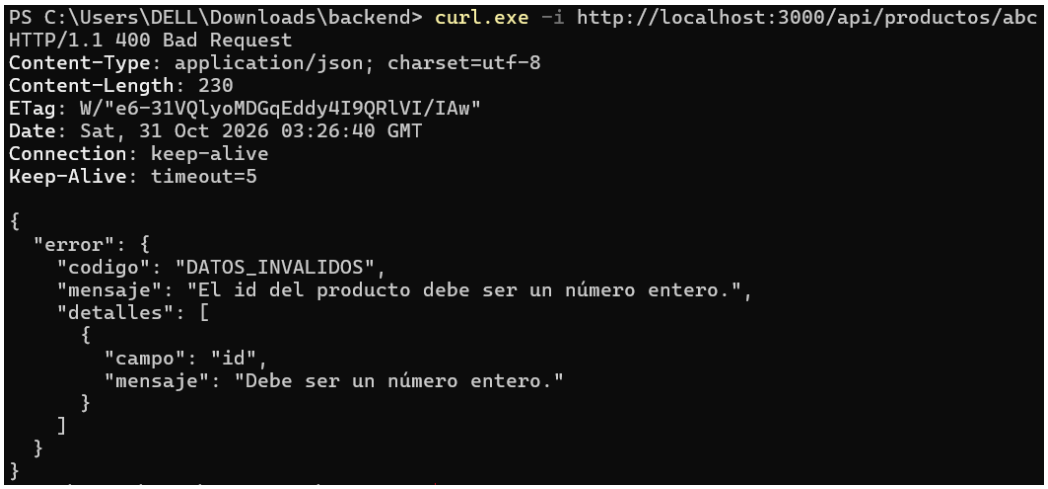


Figura 12. *GET /api/productos/abc: respuesta HTTP 400 con el código DATOS_INVALIDOS; el identificador debe ser un número entero.*

4. Interfaces y prototipos

4.1 Wireframes de las seis pantallas

Los wireframes de baja fidelidad definen las seis pantallas del sistema. Usan datos ilustrativos: los productos, precios y existencias reales son los de la base de datos (catálogo de 30 productos). Los bordes punteados naranjas distinguen las funciones deseables de lo esencial.

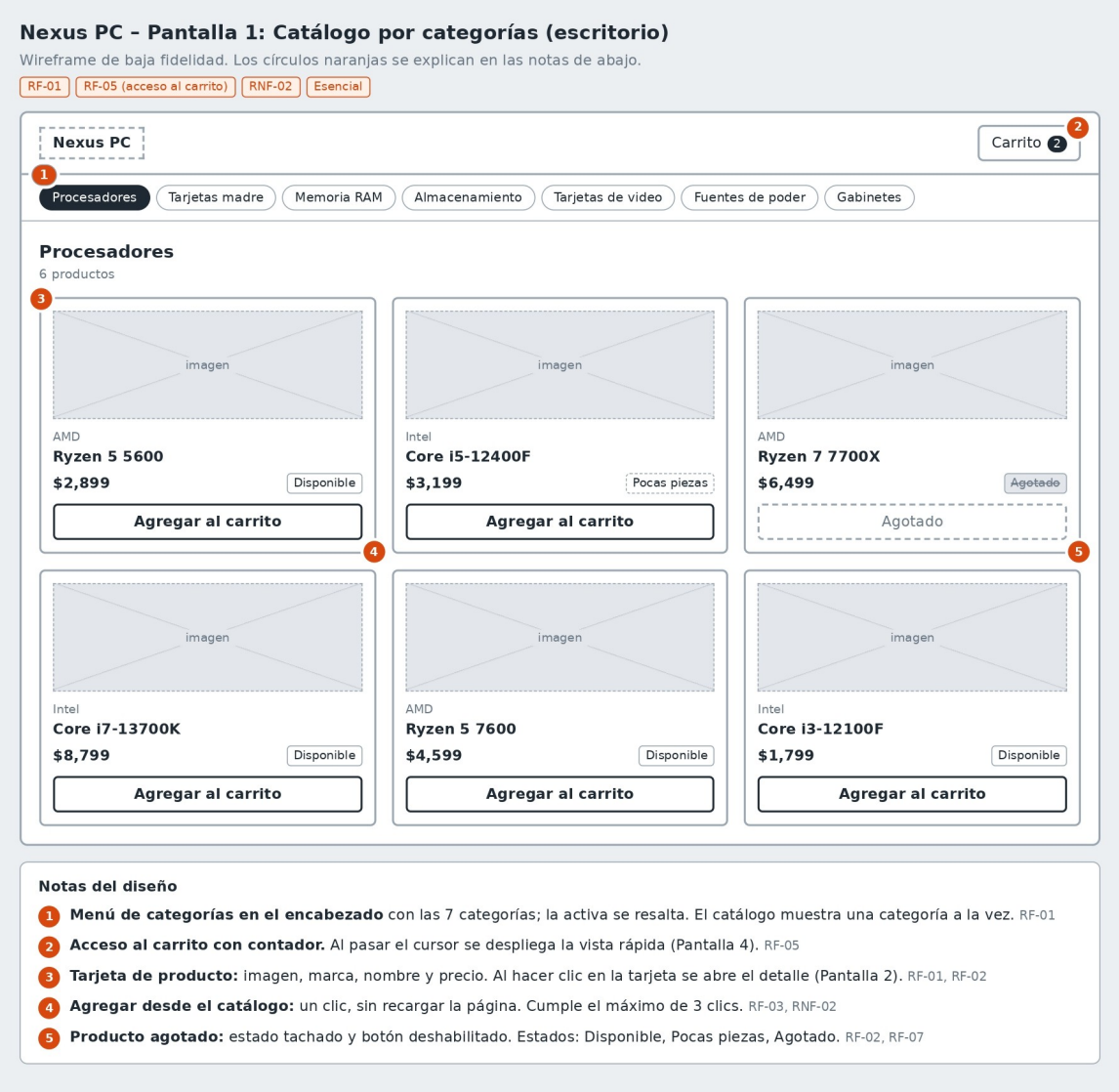


Figura 13. *Pantalla 1: catálogo por categorías (escritorio).*

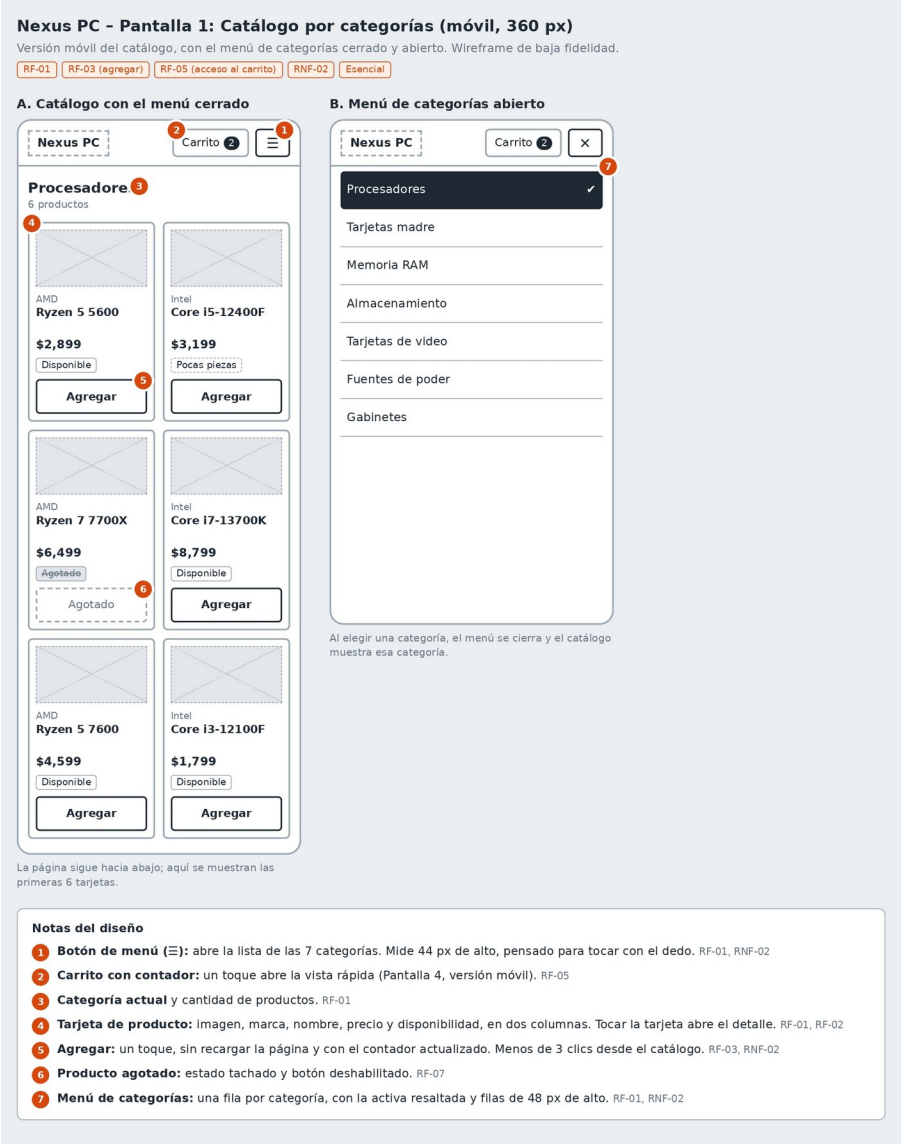


Figura 14. *Pantalla 1: catálogo por categorías (móvil de 360 px), con el menú cerrado y abierto.*

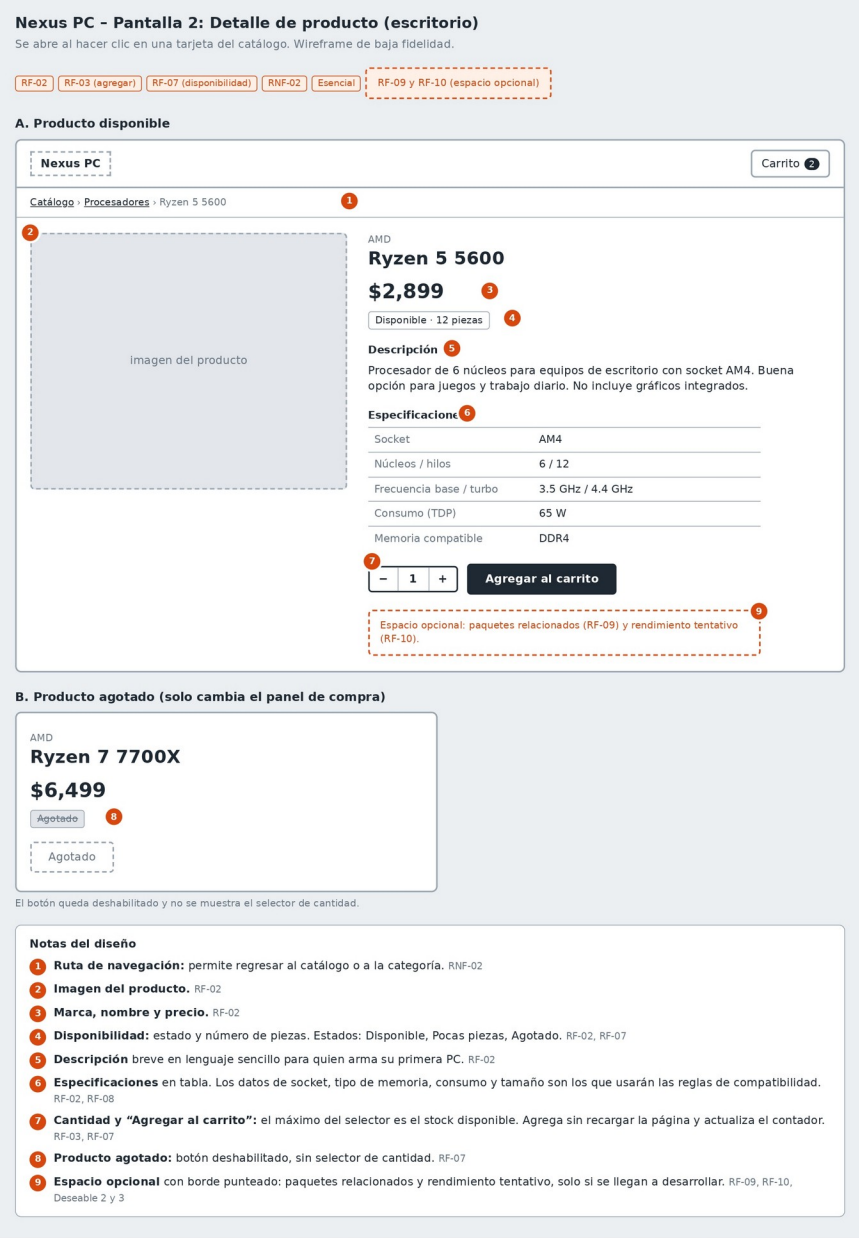


Figura 15. *Pantalla 2: detalle de producto (escritorio), con las variantes disponible y agotado.*

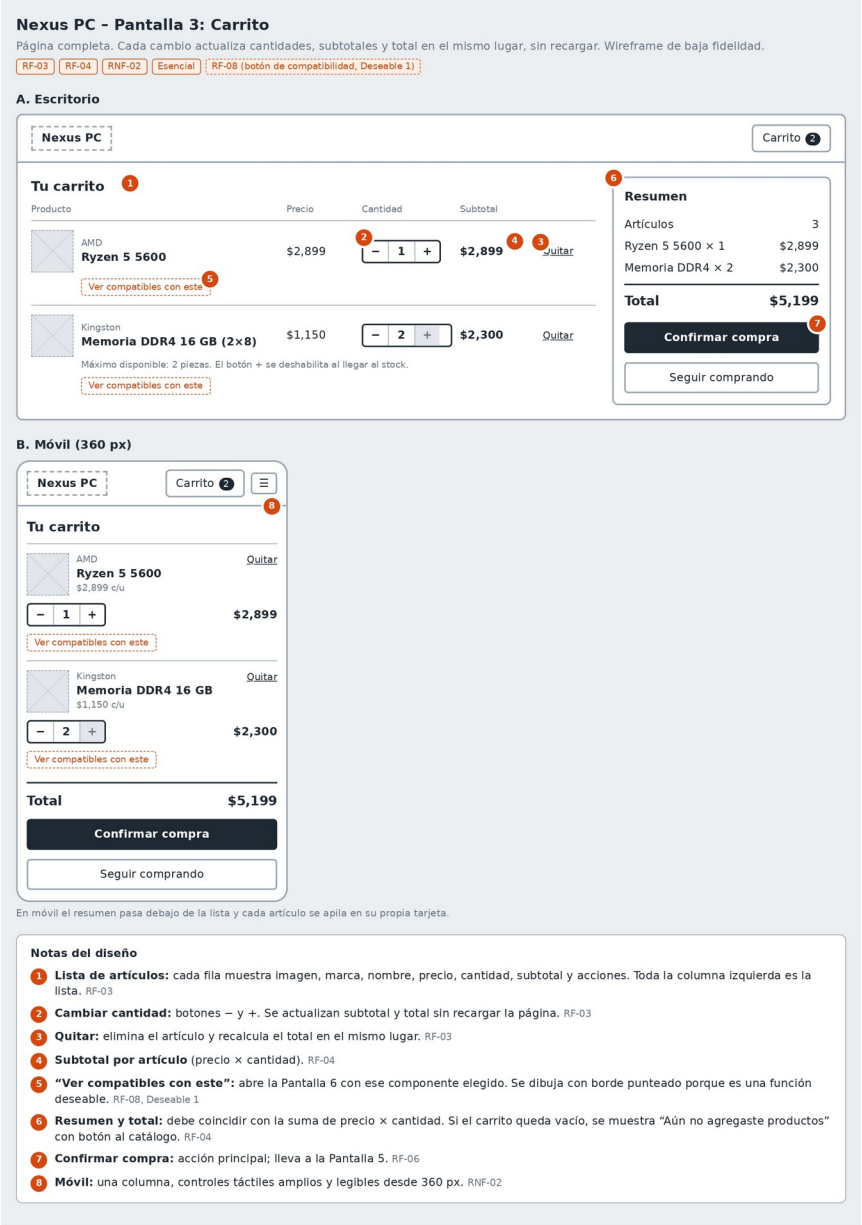


Figura 16. *Pantalla 3: carrito (escritorio y móvil).*

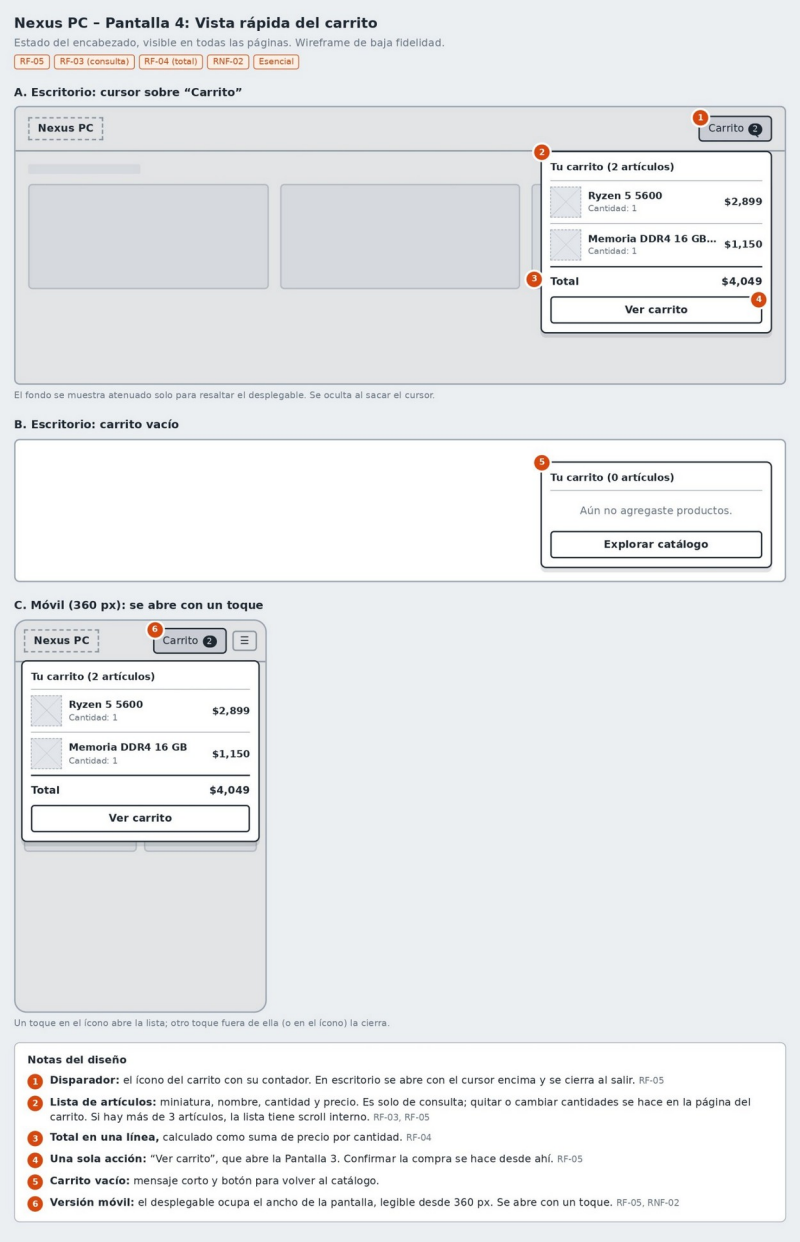


Figura 17. *Pantalla 4: vista rápida del carrito (escritorio, carrito vacío y móvil).*

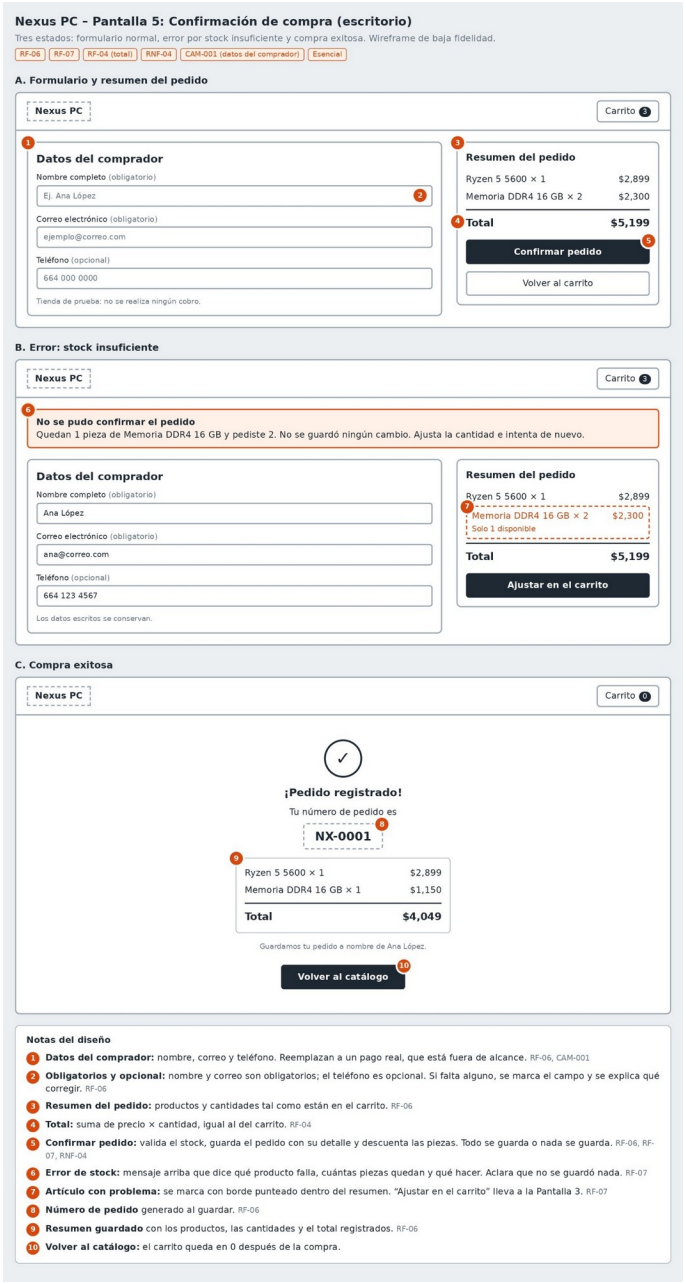


Figura 18. *Pantalla 5: confirmación de compra, con los estados de formulario, error por stock insuficiente y compra exitosa.*

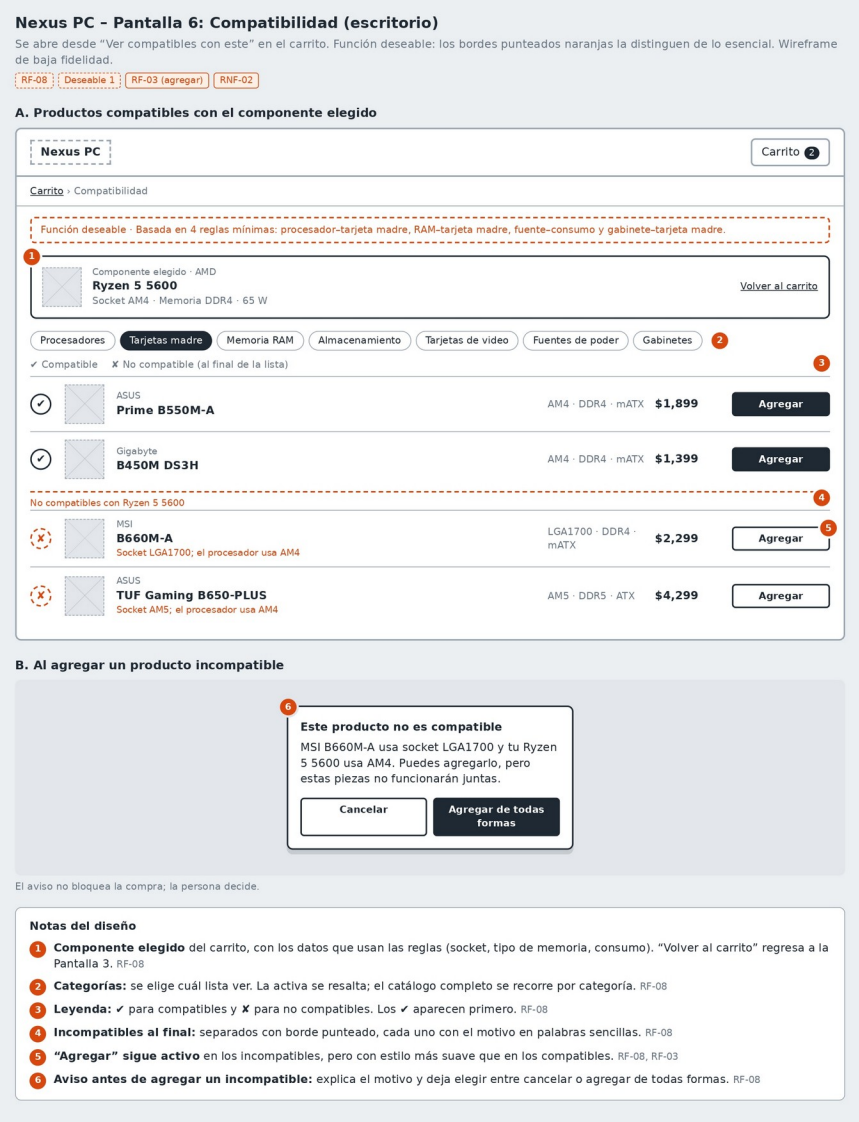


Figura 19. *Pantalla 6: compatibilidad (función deseable), con la lista de productos compatibles e incompatibles y el aviso previo a agregar.*

4.2 Interfaz funcionando con datos reales

Las capturas muestran la página en http://localhost:3000, servida por el mismo servidor que la API, con el catálogo leído de la base de datos. Corresponden a la Pantalla 1 (catálogo por categorías) y a un avance parcial de la Pantalla 4 (vista rápida del carrito).

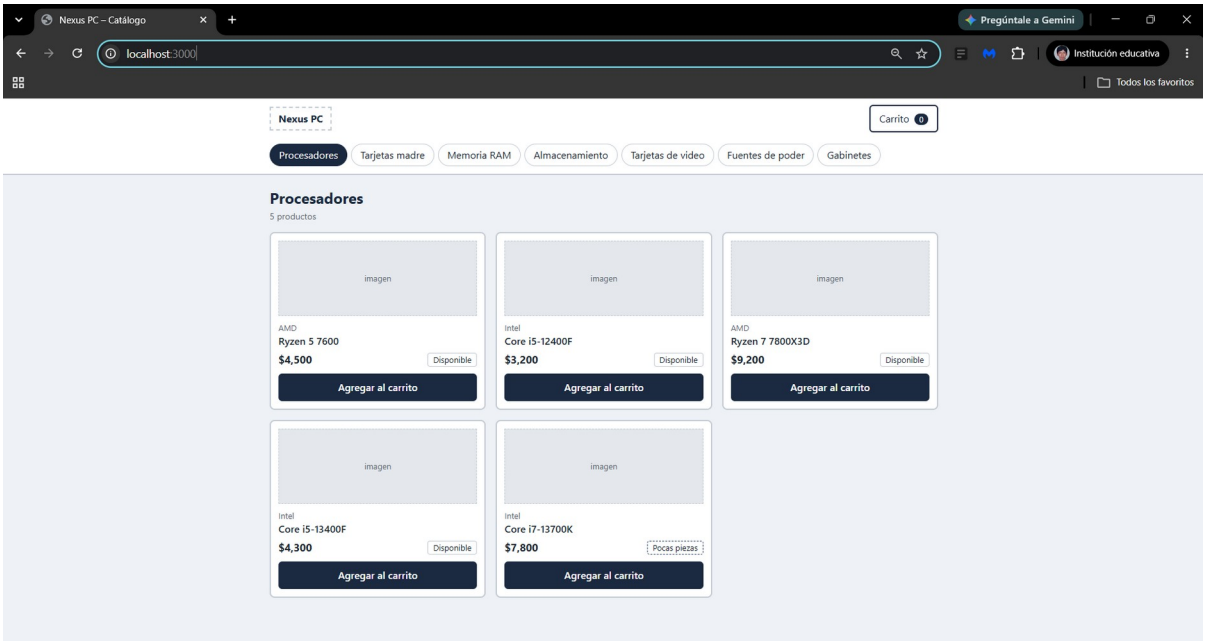


Figura 20. *Catálogo de la categoría Procesadores con datos reales: 5 productos; el Core i7-13700K aparece con el estado Pocas piezas.*

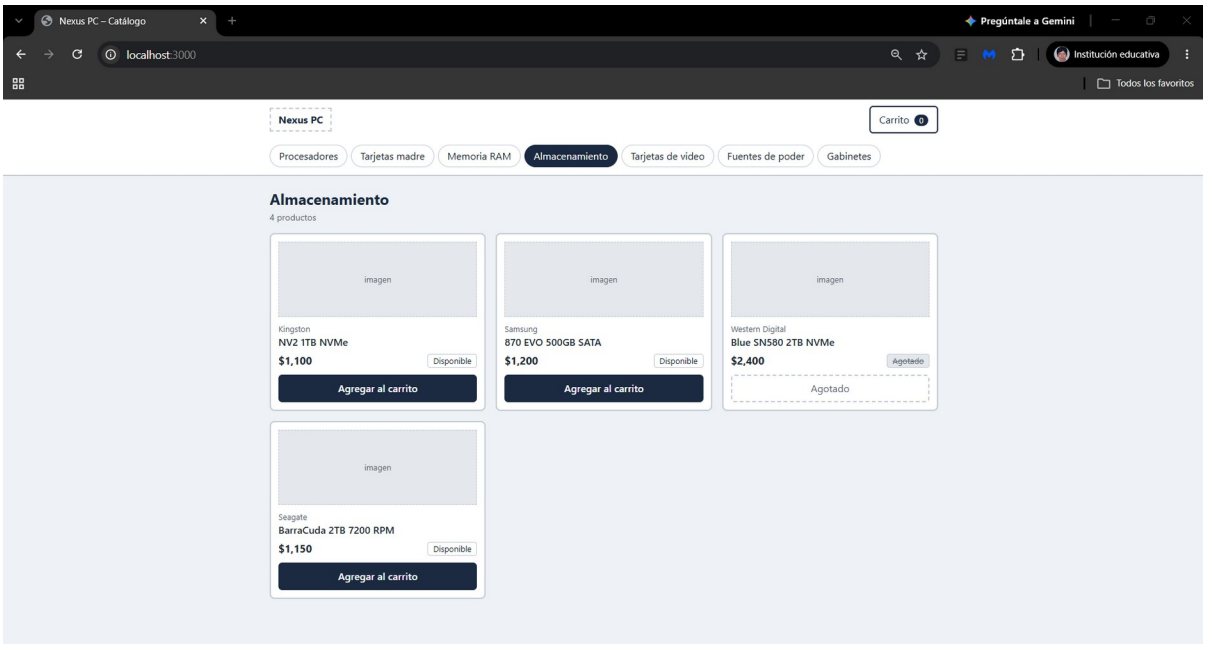


Figura 21. *Categoría Almacenamiento: 4 productos; el WD Blue SN580 2TB aparece como Agotado, con su botón deshabilitado.*

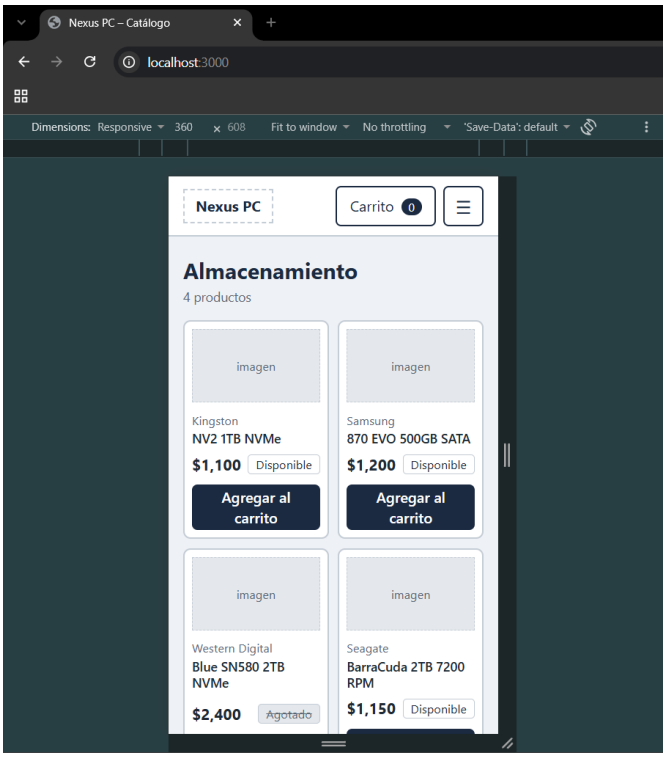


Figura 22. *Vista móvil (360 px de ancho): menú de categorías con botón, tarjetas en dos columnas y estado Agotado.*

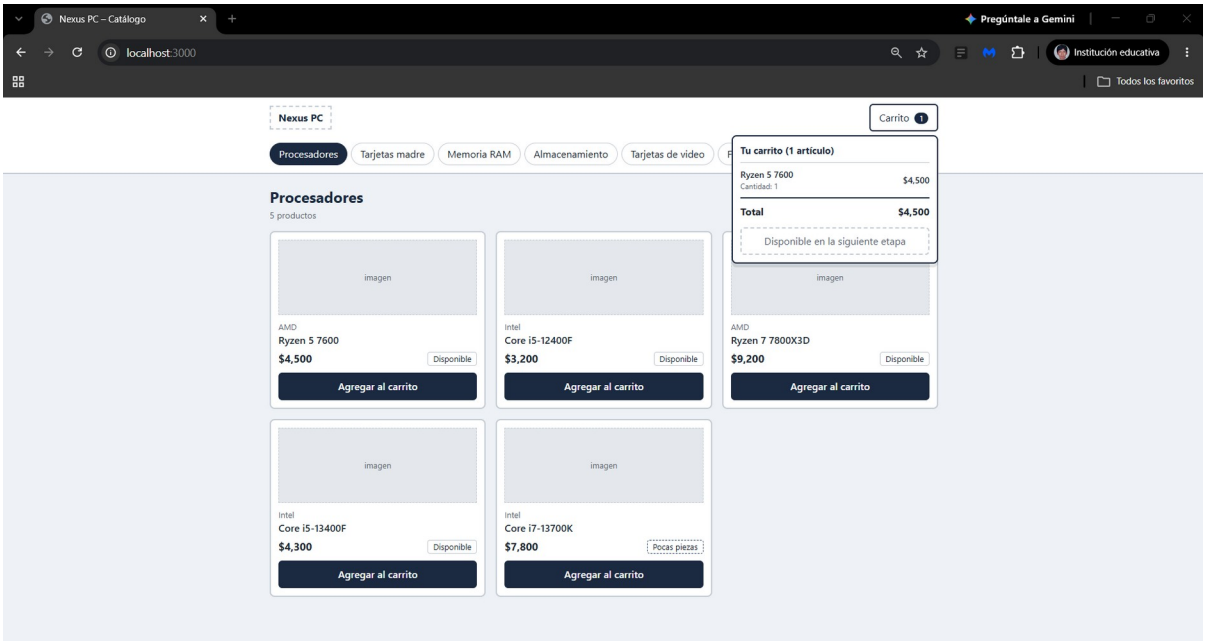


Figura 23. *Vista rápida del carrito con un producto agregado (Ryzen 5 7600, total de $4,500) y el contador del encabezado en 1. El carrito se guarda en la memoria del navegador.*

5. Diagramas y otros artefactos técnicos

Además del diagrama entidad-relación (sección 2) y de los wireframes (sección 4), el proyecto cuenta con los siguientes artefactos técnicos en el repositorio:

Artefacto	Contenido	Ubicación en el repositorio
Especificación de la API v0.2	Rutas, formato JSON, errores, reglas de pedidos y variables de entorno.	04_Documentacion/ Artefactos/
Diagrama de flujo de compra	Diseño del registro de un pedido (figura siguiente).	Especificación de la API
Plan de Calidad del Proyecto	Objetivos, métricas, control de cambios y proceso de pruebas.	04_Documentacion/
Registro de cambios	Solicitudes de cambio CAM-001 a CAM-004.	04_Documentacion/
Plan del Proyecto	Alcance, actividades, cronograma, costos, riesgos y viabilidad (documento aparte).	04_Documentacion/

La figura siguiente muestra el diseño de la ruta POST /api/pedidos, que se desarrolla en la Etapa III: valida los datos, descuenta el stock dentro de una transacción y revierte todo si alguna línea falla.

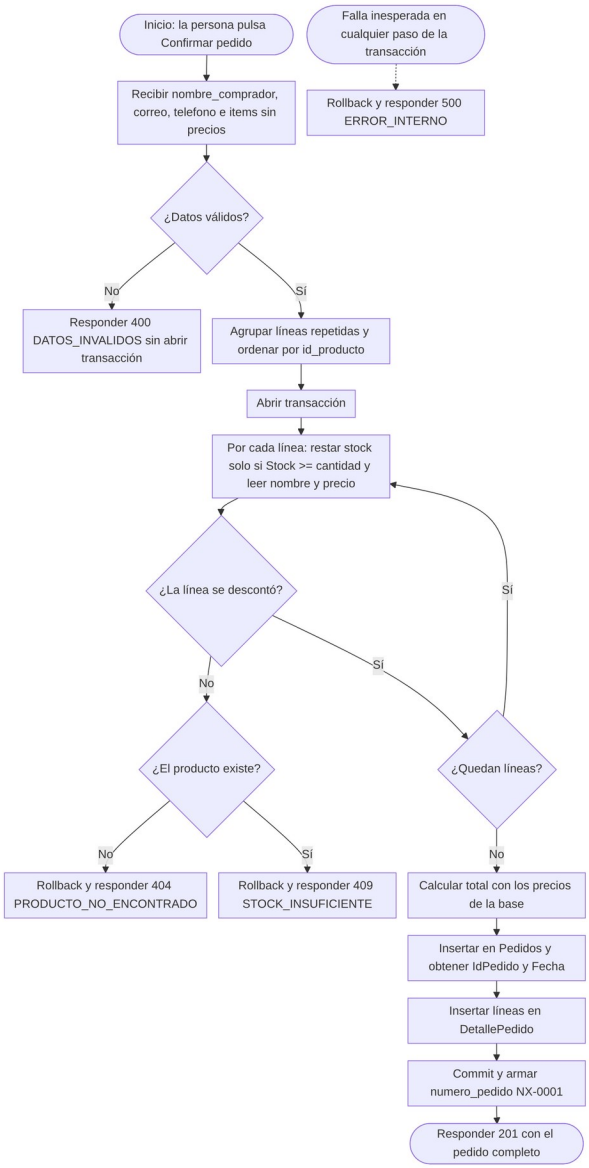


Figura 24. *Diagrama de flujo del registro de un pedido (diseño de POST /api/pedidos, pendiente de implementar en la Etapa III).*

6. Pruebas realizadas

Las pruebas de la API y de la interfaz (1 a 11) las ejecutó Omar el 30 de octubre de 2026, en su equipo, sobre el código del backend de Angel y la base de datos de Daniel. Las pruebas 12 a 14 son consultas directas a la base en SQL Server Management Studio, ejecutadas por Daniel. Entorno: Node.js 24, SQL Server 2022 Express y navegador Chrome. Solo se incluyen las pruebas que cuentan con captura.

Nº	Prueba	Resultado esperado	Resultado obtenido	Evidencia
1	Conexión a SQL Server (npm run probar-conexion)	30 productos en la base	Conexión establecida; 30 productos	Figura 6
2	Arranque del servidor (npm start)	Servidor, página y base conectados	Tres mensajes [OK]	Figura 7
3	GET /api/categorias	7 categorías con 5, 5, 4, 4, 4, 4 y 4 productos	Coincide	Figura 8
4	GET /api/productos?categoria_id=1	5 productos; el id 13 con Pocas piezas	Coincide	Figura 9
5	GET /api/productos/1	Detalle con especificaciones	Socket AM5, 65 W y 6 núcleos	Figura 10
6	GET /api/productos/999	Error 404 con formato de la especificación	HTTP 404, PRODUCTO_NO_ENCONTRADO	Figura 11
7	GET /api/productos/abc	Error 400 con formato de la especificación	HTTP 400, DATOS_INVALIDOS	Figura 12
8	Catálogo de Procesadores en la página	5 tarjetas con datos de la base	5 tarjetas; Pocas piezas en el Core i7-13700K	Figura 20
9	Catálogo de Almacenamiento en la página	4 tarjetas; producto 20 agotado	WD Blue SN580 Agotado, botón deshabilitado	Figura 21
10	Vista móvil de 360 px	Menú con botón y tarjetas en dos columnas	Coincide	Figura 22
11	Vista rápida del carrito	Producto agregado, total y contador	Ryzen 5 7600 × 1, total $4,500, contador en 1	Figura 23
12	Total de productos en la base (SSMS)	30 productos	TotalProductos = 30	Figura 3
13	Productos por categoría (SSMS)	5, 5, 4, 4, 4, 4 y 4	Coincide	Figura 4
14	Productos con stock bajo (SSMS)	Ids 13, 16 y 24 con Pocas piezas; id 20 Agotado	Coincide	Figura 5

7. Alcance actual y pendientes

7.1 Qué funciona hoy

- Base de datos v0.2 con 7 categorías y 30 productos, y usuario de acceso con permisos mínimos.
- API de consulta del catálogo (categorías, productos y detalle) con errores en formato único.
- Página con el catálogo por categorías, estados de disponibilidad y diseño adaptable a móvil.
- Carrito en memoria con vista rápida (avance parcial).

7.2 Qué sigue en la Etapa III

- Detalle de producto en la interfaz (UI3) y carrito completo con cantidades (UI4).
- Registro de pedidos: ruta POST /api/pedidos (API3) y pantalla de confirmación de compra (UI5).
- Compatibilidad entre componentes (API4 y UI6), como primera función deseable.
- Paquetes, administración y cuentas de usuario, sujetos a la revisión de alcance del 3 de noviembre.